Programação Web

Ficha de Exercícios Práticos

Introdução ao React

Componentes · Navegação · Inputs · Estado

Objetivos de Aprendizagem

Ao concluir esta ficha, o aluno deverá ser capaz de:

- Criar e estruturar componentes funcionais em React
- Utilizar JSX para renderizar elementos e listas
- Implementar navegação entre páginas com React Router
- Gerir estado local com o hook useState
- Criar formulários controlados e comunicar entre componentes via props

Parte I — React Básico	Parte II — Navegação	Parte III — Inputs	Total
/25	/25	/50	**/100**

Parte I — Introdução ao React: Componentes e JSX

Nesta secção vais criar os teus primeiros componentes React, compreender JSX e trabalhar com renderização dinâmica de listas.

Exercício 1 O Meu Primeiro Componente — Guiado (10 pts)

Contexto

Os componentes são os blocos fundamentais do React. Um componente funcional é uma função JavaScript que devolve JSX — uma sintaxe parecida com HTML que descreve o que deve aparecer no ecrã.

Recorda — Node.js e REST API

Assim como criaste funções em Node.js para tratar pedidos HTTP, em React crias funções (componentes) para renderizar partes da interface. A lógica é modular nos dois casos!

Configuração do Projeto

Antes de começar, cria um novo projeto React com Vite:

```
# Criar projeto
npm create vite@latest ficha-react -- --template react
cd ficha-react
npm install
npm run dev
```

Dica

O Vite inicia um servidor de desenvolvimento em http://localhost:5173. Mantém o terminal aberto enquanto trabalhas!

Tarefa 1.1 — Componente de Cartão de Utilizador

Abre o ficheiro src/App.jsx. Encontrarás o seguinte código base — lê-o com atenção:

```
// src/App.jsx
function CartaoUtilizador({ nome, curso, ano }) {
  return (
    <div className="cartao">
      {/* TODO 1: Adiciona um <h2> com o nome do utilizador */}

      {/* TODO 2: Adiciona um <p> com o curso */}

      {/* TODO 3: Adiciona um <p> com o texto: "Ano: " seguido do valor de ano
*/}

    </div>
```

```
  );
}

export default function App() {
  return (
    <div>
      {/* TODO 4: Usa o componente CartaoUtilizador com os teus dados reais */}
      {/* Dica: <CartaoUtilizador nome="..." curso="..." ano={1} /> */}

    </div>
  );
}
```

Passos a Seguir

1. Substitui os comentários TODO 1, 2 e 3 pelos elementos JSX corretos.
2. No TODO 4, instancia o componente CartaoUtilizador passando o teu nome, curso e ano como props.
3. Adiciona o seguinte CSS no ficheiro src/index.css para estilizar o cartão:

```
/* src/index.css — adiciona no final */
.cartao {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  max-width: 300px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
```

Tarefa 1.2 — Lista de Estudantes

Agora vais renderizar uma lista de vários estudantes usando .map(). Completa o código abaixo:

```
// Adiciona abaixo do componente CartaoUtilizador em App.jsx

const estudantes = [
  { id: 1, nome: "Ana Costa",    curso: "Eng. Informática", ano: 1 },
  { id: 2, nome: "Bruno Silva",  curso: "Eng. Informática", ano: 1 },
  { id: 3, nome: "Carla Mendes", curso: "Ciências da Computação", ano: 2 },
];

function ListaEstudantes() {
  return (
    <div>
      <h1>Turma de Programação Web</h1>
      {/* TODO: Usa .map() para renderizar um CartaoUtilizador por cada
estudante */}
      {/* IMPORTANTE: Não te esqueças da prop key={estudante.id} */}

    </div>
  );
}
```

Atenção — A prop key

Sempre que renderizas uma lista com .map() em React, deves fornecer uma prop key única a cada elemento. Sem ela, o React não consegue identificar quais elementos mudaram e mostra um aviso no browser.

Resultado Esperado

A página deve mostrar 3 cartões de estudantes lado a lado, cada um com nome, curso e ano.

Exercício 2 Renderização Condicional — Desafio (15 pts)

Partindo da lista de estudantes do Exercício 1, adiciona uma funcionalidade de destaque para alunos de 1.º ano.

O que deves implementar

- Adiciona uma prop booleana destaque ao componente CartaoUtilizador.
- Quando destaque for true, o cartão deve exibir um badge/etiqueta com o texto "Caloiro" em destaque visual (cor diferente, por exemplo).
- Quando destaque for false ou não definida, o badge não deve aparecer.
- Na ListaEstudantes, passa destaque={estudante.ano === 1} para cada cartão.

Dica — Renderização Condicional

Podes usar o operador ternário dentro do JSX: {condicao ? <Elemento /> : null}. Ou ainda o operador &&: {condicao && <Elemento />}

Código de Apoio (Ponto de Partida)

```
function CartaoUtilizador({ nome, curso, ano, destaque }) {
  return (
    <div className={`cartao ${destaque ? 'cartao-destaque' : ''}`}>
      {/* TODO: Renderiza o badge 'Caloiro' APENAS quando destaque for true */}

      <h2>{nome}</h2>
      <p>{curso}</p>
      <p>Ano: {ano}</p>
    </div>
  );
}
```

CSS Sugerido

```
.cartao-destaque { border-color: #2980B9; border-width: 2px; }
.badge {
  background: #2980B9; color: white;
  padding: 2px 8px; border-radius: 12px;
  font-size: 12px; font-weight: bold;
}
```

Parte II — Navegação com React Router

Nesta secção vais estruturar a aplicação em múltiplas páginas usando o React Router DOM, a biblioteca mais popular para navegação em aplicações React SPA (Single Page Application).

Exercício 3 Configurar Rotas Básicas — Guiado (10 pts)

Instalação

Instala a biblioteca de routing no projeto:

```
npm install react-router-dom
```

Recorda — Express Routes vs React Router

No Express definiste rotas como app.get('/posts', handler). No React Router a lógica é parecida, mas do lado do cliente (browser): defines que componente renderizar para cada URL, sem precisar de servidor!

Estrutura de Ficheiros

Cria a seguinte estrutura dentro de src/:

```
src/
  pages/
    Home.jsx
    Sobre.jsx
    Estudantes.jsx
  components/
    Navbar.jsx
  App.jsx
```

Tarefa 3.1 — Configurar o Router

Substitui o conteúdo do App.jsx pelo seguinte código base e completa os TODOs:

```
// src/App.jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import Sobre from './pages/Sobre';
import Estudantes from './pages/Estudantes';

export default function App() {
  return (
    <BrowserRouter>
      {/* TODO 1: Coloca o componente Navbar aqui, fora das Routes */}

      <Routes>
        {/* TODO 2: Define rota para '/' que renderiza <Home /> */}
```

```
        {/* TODO 3: Define rota para '/sobre' que renderiza <Sobre /> */}

        {/* TODO 4: Define rota para '/estudantes' que renderiza <Estudantes />
*/}

      </Routes>
    </BrowserRouter>
  );
}
```

Tarefa 3.2 — Criar a Navbar

Cria o componente Navbar.jsx com links de navegação:

```
// src/components/Navbar.jsx
import { NavLink } from 'react-router-dom';

export default function Navbar() {
  return (
    <nav style={{ display: 'flex', gap: '16px', padding: '12px', background:
'#1E3A5F' }}>
      {/* TODO: Adiciona 3 NavLinks para '/', '/sobre', '/estudantes' */}
      {/* Dica: <NavLink to="/" style={({ isActive }) => ({ color: isActive ?
'#61DAFB' : 'white' })}>Início</NavLink> */}

    </nav>
  );
}
```

Tarefa 3.3 — Criar as Páginas

Cria os três ficheiros de página com conteúdo simples (podes usar o seguinte como ponto de partida):

```
// src/pages/Home.jsx
export default function Home() {
  return (
    <main style={{ padding: '24px' }}>
      <h1>Bem-vindo ao Portal da Turma</h1>
      {/* TODO: Adiciona um parágrafo descritivo e uma lista com 3
funcionalidades do portal */}

    </main>
  );
}
```

```
// src/pages/Sobre.jsx
export default function Sobre() {
  return (
    <main style={{ padding: '24px' }}>
      <h1>Sobre Este Projeto</h1>
      {/* TODO: Adiciona informação sobre a disciplina e os objetivos */}
```

```
    </main>
  );
}
```

```
// src/pages/Estudantes.jsx
// TODO: Importa e utiliza aqui o componente ListaEstudantes do Exercício 1
export default function Estudantes() {
  return (
    <main style={{ padding: '24px' }}>
      <h1>Estudantes</h1>
      {/* TODO: Renderiza a lista de estudantes aqui */}

    </main>
  );
}
```

Exercício 4 Rota Dinâmica — Perfil do Estudante

Desafio (15 pts)

Adiciona uma rota dinâmica que mostra o perfil detalhado de cada estudante ao clicar no respetivo cartão.

O que deves implementar

- Cria uma nova página src/pages/PerfilEstudante.jsx.
- Define uma rota dinâmica /estudantes/:id no App.jsx.
- Na página de Estudantes, torna cada cartão clicável (usa o componente Link do react-router-dom).
- Na página PerfilEstudante, usa o hook useParams para ler o id da URL e mostrar os detalhes do estudante correspondente.

Dica — useParams

import { useParams } from 'react-router-dom'; const { id } = useParams(); O valor de id corresponde ao segmento dinâmico definido na rota como :id.

Código de Apoio

```
// src/pages/PerfilEstudante.jsx
import { useParams, Link } from 'react-router-dom';

// Exporta o array de estudantes de um ficheiro separado ou importa daqui:
const estudantes = [
  { id: 1, nome: "Ana Costa",    curso: "Eng. Informática", ano: 1, email:
"ana@uni.pt" },
  { id: 2, nome: "Bruno Silva",  curso: "Eng. Informática", ano: 1, email:
"bruno@uni.pt" },
  { id: 3, nome: "Carla Mendes", curso: "Ciências da Computação", ano: 2,
email: "carla@uni.pt" },
];

export default function PerfilEstudante() {
  const { id } = useParams();
```

```
  // TODO 1: Encontra o estudante cujo id corresponde ao parâmetro da URL
  // Atenção: useParams devolve strings, o id no array é number — converte!
  const estudante = null; // substitui isto

  // TODO 2: Se o estudante não existir, mostra uma mensagem de erro
  // Dica: if (!estudante) return <p>Estudante não encontrado.</p>

  return (
    <main style={{ padding: '24px' }}>
      {/* TODO 3: Mostra o nome, curso, ano e email do estudante */}

      {/* TODO 4: Adiciona um Link para voltar à lista de estudantes */}

    </main>
  );
}
```

Parte III — Inputs, Estado e Comunicação entre Componentes

Nesta última secção trabalhas com formulários controlados, o hook useState, e a passagem de dados entre componentes pai e filho através de props e callbacks.

Exercício 5 Formulário Controlado com useState — Guiado (15 pts)

Recorda — Estado em React

Em Node.js usavas variáveis no servidor para guardar estado. Em React, o hook useState permite guardar estado local num componente — sempre que o estado muda, o React re-renderiza o componente automaticamente.

Tarefa 5.1 — Formulário de Registo

Cria o ficheiro src/pages/Registo.jsx. Completa os TODOs abaixo:

```
// src/pages/Registo.jsx
import { useState } from 'react';

export default function Registo() {
  // TODO 1: Declara 3 variáveis de estado com useState:
  //   - nome (string vazia)
  //   - email (string vazia)
  //   - ano (número 1)



  // TODO 2: Cria a função handleSubmit que:
  //   a) Previne o comportamento padrão do formulário
  //   b) Mostra um alert com os dados submetidos (JSON.stringify)
  //   c) Limpa os campos do formulário
  function handleSubmit(e) {

  }

  return (
    <main style={{ padding: '24px', maxWidth: '500px' }}>
      <h1>Registo de Estudante</h1>

      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '12px' }}>
          <label>Nome:</label>
          {/* TODO 3: Input controlado — value={nome} e onChange para atualizar
nome */}
          <input
            type="text"
            placeholder="O teu nome"
            style={{ display: 'block', width: '100%', padding: '8px',
marginTop: '4px' }}
          />
        </div>

        <div style={{ marginBottom: '12px' }}>
          <label>Email:</label>
          {/* TODO 4: Input controlado para email */}
```

```
          <input
            type="email"
            placeholder="email@exemplo.pt"
            style={{ display: 'block', width: '100%', padding: '8px',
marginTop: '4px' }}
          />
        </div>

        <div style={{ marginBottom: '16px' }}>
          <label>Ano Curricular:</label>
          {/* TODO 5: Select controlado para o ano (opções: 1, 2, 3) */}
          <select style={{ display: 'block', width: '100%', padding: '8px',
marginTop: '4px' }}>
            <option value={1}>1.º Ano</option>
            <option value={2}>2.º Ano</option>
            <option value={3}>3.º Ano</option>
          </select>
        </div>

        <button type="submit" style={{ padding: '10px 24px', background:
'#1E3A5F', color: 'white', border: 'none', borderRadius: '4px', cursor:
'pointer' }}>
          Registar
        </button>
      </form>

      {/* TODO 6: Mostra abaixo do formulário uma pré-visualização em tempo
real */}
      {/* Exemplo: "Pré-visualização: Ana Costa | ana@uni.pt | Ano 1" */}
      {nome && (
        <div style={{ marginTop: '16px', padding: '12px', background:
'#F2F3F4', borderRadius: '4px' }}>
          {/* Mostra os dados aqui */}
        </div>
      )}
    </main>
  );
}
```

Passos a Seguir

1. Adiciona a rota /registo no App.jsx.
2. Adiciona o link 'Registo' na Navbar.
3. Testa o formulário: os campos devem atualizar a pré-visualização em tempo real ao escrever.

Exercício 6 Comunicação Pai-Filho com Props e Callbacks — Desafio (15 pts)

Neste desafio vais implementar comunicação bidirecional entre componentes: pai envia dados ao filho via props, filho comunica eventos ao pai via funções callback.

O que deves implementar

Cria uma página src/pages/Notas.jsx que permita gerir notas de estudantes:

- Componente pai Notas — mantém o estado global (lista de notas).

- Componente filho FormularioNota — recebe uma função onAdicionarNota via props e chama-a quando o formulário é submetido.
- Componente filho TabelaNotas — recebe o array de notas via props e renderiza-o numa tabela.

Dica — Padrão de Comunicação

Pai -> Filho: passa dados via props | Filho -> Pai: o pai passa uma função como prop; o filho chama essa função com os dados. Este padrão é equivalente a emitir eventos num backend com EventEmitter em Node.js!

Código de Apoio

```
// src/pages/Notas.jsx
import { useState } from 'react';

// Componente filho: formulário para adicionar nota
function FormularioNota({ onAdicionarNota }) {
  const [estudante, setEstudante] = useState('');
  const [nota, setNota] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    // TODO 1: Valida que estudante e nota não estão vazios
    // TODO 2: Valida que nota está entre 0 e 20
    // TODO 3: Chama onAdicionarNota({ estudante, nota: Number(nota) })
    // TODO 4: Limpa os campos
  }

  return (
    <form onSubmit={handleSubmit} style={{ display: 'flex', gap: '8px',
marginBottom: '16px' }}>
      <input value={estudante} onChange={e => setEstudante(e.target.value)}
        placeholder="Nome do estudante" style={{ padding: '8px', flex: 2 }} />
      <input value={nota} onChange={e => setNota(e.target.value)}
        type="number" min="0" max="20" placeholder="Nota (0-20)" style={{
padding: '8px', flex: 1 }} />
      <button type="submit" style={{ padding: '8px 16px', background:
'#1E8449', color: 'white', border: 'none', borderRadius: '4px' }}>
        Adicionar
      </button>
    </form>
  );
}

// Componente filho: tabela de notas
function TabelaNotas({ notas }) {
  // TODO 5: Renderiza uma tabela HTML com colunas: Estudante | Nota |
Classificação
  // TODO 6: Classifica a nota: >= 18 = Excelente, >= 14 = Bom, >= 10 =
Suficiente, < 10 = Insuficiente
  // TODO 7: Mostra a classificação em cor diferente
(verde/amarelo/laranja/vermelho)
  return (
    <table style={{ width: '100%', borderCollapse: 'collapse' }}>
      <thead>
        <tr style={{ background: '#1E3A5F', color: 'white' }}>
          <th style={{ padding: '10px', textAlign: 'left' }}>Estudante</th>
          <th style={{ padding: '10px', textAlign: 'center' }}>Nota</th>
```

```
          <th style={{ padding: '10px', textAlign: 'center'
}}>Classificação</th>
        </tr>
      </thead>
      <tbody>
        {/* TODO: .map() das notas */}
      </tbody>
    </table>
  );
}

// Componente pai: gere o estado e coordena os filhos
export default function Notas() {
  // TODO 8: Declara estado para o array de notas (começa com array vazio)
  const [notas, setNotas] = useState([]);

  // TODO 9: Implementa a função adicionarNota que recebe { estudante, nota }
  //   e adiciona ao array de notas (usa spread: [...notas, novaNota])
  function adicionarNota(novaNota) {

  }

  return (
    <main style={{ padding: '24px' }}>
      <h1>Gestão de Notas</h1>
      {/* TODO 10: Passa adicionarNota como prop onAdicionarNota ao
FormularioNota */}
      <FormularioNota onAdicionarNota={adicionarNota} />
      {/* TODO 11: Passa notas ao TabelaNotas */}
      <TabelaNotas notas={notas} />
      {/* TODO 12 (Bónus): Mostra a média das notas abaixo da tabela */}
    </main>
  );
}
```

Mini-Projeto Integrador — Portal da Turma

Exercício Final (50 pontos)

Este mini-projeto integra todos os conceitos trabalhados nos exercícios anteriores.
Podes usar código desenvolvido nas partes anteriores como ponto de partida.
Prazo de entrega: consulta o Moodle da disciplina.

Desenvolve uma aplicação React completa chamada Portal da Turma. A aplicação deve ter pelo menos 4 páginas funcionais, navegação por React Router e gestão de estado local.

Estrutura Obrigatória da Aplicação

Página	Requisitos Mínimos
`/ (Início)`	Apresentação do portal; lista de destaques ou avisos; pelo menos um componente reutilizável.
`/estudantes`	Lista de estudantes com filtro por nome (input de pesquisa em tempo real com useState).
`/estudantes/:id`	Perfil detalhado de um estudante (useParams); botão para voltar à lista.
`/notas`	Formulário para adicionar notas; tabela com notas e classificação; média calculada dinamicamente.

Requisitos Técnicos

- Mínimo de 5 componentes funcionais separados (para além das páginas).
- Utilização de pelo menos 3 instâncias de useState.
- Comunicação pai-filho com props em pelo menos um par de componentes.
- Navegação com NavLink (incluindo destaque da rota ativa na Navbar).
- Pelo menos uma rota dinâmica com useParams.
- Código organizado em pastas pages/ e components/.

Critérios de Avaliação

Critério	Pontuação	Obtido
Estrutura de componentes e organização do código	10	
Funcionalidade de routing (navegação + rotas dinâmicas)	10	
Gestão de estado (useState) e formulários controlados	10	
Comunicação entre componentes (props e callbacks)	10	
Qualidade e usabilidade da interface	5	
Funcionalidades extra (bónus)	5	
TOTAL	**50**	

Funcionalidades Bónus (até +5 pontos)

- Persistência dos dados com localStorage (os dados mantêm-se ao refrescar a página).
- Filtro/pesquisa de notas por nome de estudante.
- Ordenação da tabela de notas (crescente / decrescente) ao clicar no cabeçalho.
- Página 404 personalizada para rotas não definidas (usa <Route path="*">).

Entrega

Submete o projeto comprimido em .zip no Moodle, excluindo a pasta node_modules/.
O projeto deve arrancar com npm install seguido de npm run dev sem erros.
Inclui um ficheiro README.md com o teu nome, número e instruções de execução.

Referência Rápida — Conceitos Essenciais

Hooks Utilizados Nesta Ficha

Hook	Uso	Exemplo
`useState`	Estado local num componente	`const [n, setN] = useState('')`
`useParams`	Lê parâmetros dinâmicos da URL	`const { id } = useParams()`
`NavLink / Link`	Navegação sem recarregar a página	`<NavLink to="/">Início</NavLink>`

Anatomia de um Componente Funcional

```
// Importações no topo
import { useState } from 'react';

// Componente = função que retorna JSX
export default function MeuComponente({ prop1, prop2 }) {
  // 1. Hooks (sempre no topo, nunca dentro de if/for)
  const [estado, setEstado] = useState(valorInicial);

  // 2. Funções/handlers
  function handleClick() {
    setEstado(novoValor);
  }

  // 3. JSX retornado (sempre um único elemento raiz)
  return (
    <div>
      <p>{prop1}</p>
      <button onClick={handleClick}>{estado}</button>
    </div>
  );
}
```

Boas Práticas

Nomeia sempre os componentes com PascalCase (ex: CartaoUtilizador). Usa camelCase para variáveis, funções e props. Um componente deve fazer apenas uma coisa — se ficar grande demais, divide-o em sub-componentes.